

分布式多人实时协同绘画工 具

需求规格说明书

(Software Requirements Specification)

设计题目：分布式多人实时协同绘画工具

专业方向：软件工程

2026 年 5 月

目录

第一章 引言	1
1.1 编写目的	1
1.2 项目背景与动机	1
第二章 总体描述	3
2.1 系统愿景与目标	3
2.2 用户特征与角色	3
2.3 运行环境与技术栈约束	4
第三章 功能性需求	5
3.1 房间与会话管理域	5
3.1.1 房间创建与配置	5
3.1.2 用户加入与路由分配	5
3.2 全局画作同步域 (Snapshot & Replay) ——核心功能	6
3.2.1 新用户入房状态同步	6
3.2.2 操作日志持久化	6
3.3 实时协同绘画域 (Real-time Drawing) ——核心功能	7
3.3.1 笔触渲染	7
3.3.2 光标同步 (User Presence)	7
3.3.3 操作广播	7
3.3.4 功能需求点追踪	8

目录	II
第四章 非功能性需求	9
4.1 性能与吞吐量需求 (Performance)	9
4.2 可用性与容错需求 (Availability & Fault Tolerance)	10
4.2.1 断线重连机制	10
4.2.2 节点故障隔离	10
第五章 接口设计规范	11
5.1 HTTP RESTful 接口	11
5.2 WebSocket 事件交互协议	12
5.2.1 上行事件 (Client → Server)	12
5.2.2 下行事件 (Server → Client)	12
5.2.3 心跳包 (Ping/Pong) 维持长连接机制	13
第六章 数据流与质量保障	14
6.1 数据完整性保障	14
6.2 安全性需求	14

第一章 引言

1.1 编写目的

本文档《分布式多人实时协同绘画工具需求规格说明书》(Software Requirements Specification, 简称 SRS) 旨在以严谨的学术化语言, 明确界定在线协作画板系统的功能边界、性能约束与质量属性, 为后续微服务架构开发、全链路压测基准确立以及项目验收评审提供无可争议的基准文档。

具体而言, 本文档承担以下职责: 其一, 精确描述系统的功能性需求 (Functional Requirements), 以用例 (Use Case) 与功能需求点追踪矩阵的形式, 清晰划分每一项功能的输入、处理逻辑与输出; 其二, 完整定义系统的非功能性需求 (Non-Functional Requirements), 涵盖性能指标 (如 QPS、延迟 P99)、可用性等级、容错策略与可观测性要求; 其三, 建立需求变更的追踪基线, 确保后续迭代开发中每一行代码均可回溯至具体的文档条款。通过上述三条线索的并行推进, 本文档为整个项目生命周期中的需求管理活动提供了制度化的锚点。

1.2 项目背景与动机

传统单体架构的在线画板系统, 在面向单一用户或少量并发用户的场景下尚能正常运作。然而, 当系统需要支持**多人高频涂抹协作**时, 一系列深层次的架构缺陷将逐一暴露:

- **WebSocket 丢包导致笔触断层:** 在单体架构中, 所有 WebSocket 连接均承载于单一服务器进程。一旦网络出现瞬时抖动或服务器进程发生 Full GC 停顿, 正在传输中的增量坐标包将被丢弃。由于缺乏序列号 (Sequence ID) 机制与重传补偿逻辑, 接收端无法感知丢包, 画布上便会出现令人困扰的笔触断层现象。

- **数据库写风暴：**当数十名用户同时在画布上作画时，每一帧增量坐标都可能触发一次数据库写操作。以鼠标移动事件每 16ms 触发一次计算，每位用户每秒产生约 60 次写入，100 名用户即意味着 6000 次/秒的数据库写入。在单体架构下，这类写风暴将直接击穿数据库连接池，造成全局性的服务降级。
- **跨地域笔触不同步：**分布式部署的首要挑战在于网络延迟的不对称性。北京用户与深圳用户若连接到不同的接入节点，由于缺乏统一的操作序列化和冲突仲裁机制，两地用户在同一时间窗口内绘制的线条可能出现不可预期的穿插与覆盖。
- **单点故障导致全局中断：**单体服务器的崩溃将导致所有在线用户会话瞬间中断，已绘制但未持久化的数据将全部丢失。

鉴于上述痛点，本项目提出**分布式协同绘画系统改造方案**：引入 WebSocket 网关集群实现水平扩展；采用 Redis Pub/Sub + Kafka 实现跨节点消息路由；引入 Event Sourcing 架构，以不可变操作日志流替代可变位图状态，彻底消解并发写入冲突；最终通过 CRDT（无冲突复制数据类型）思想确保最终一致性。

第二章 总体描述

2.1 系统愿景与目标

本项目的核心愿景是构建一款支持数百人同房间协作、笔触毫秒级同步、支持无限回放的分布式在线画板系统。具体目标分解如下：

- **高并发接入：**单房间最高支持同时 200 名协作者（含绘画者与观战者），全局系统支持 10,000+ 并发 WebSocket 连接。
- **实时性保证：**笔触广播端到端延迟 $P99 \leq 50\text{ms}$ ，确保异地用户感知到的绘制流畅度接近本地应用。
- **一致性保障：**通过服务器单调递增序列号（Monotonically Increasing Sequence ID）序列化所有操作，确保所有客户端最终收敛至同一画布状态。
- **持久化与回放：**完整记录所有操作日志，支持任意历史时刻画布状态的精确重建与离线回放。
- **容错可用性：**任意单个 WebSocket 网关节点宕机时，客户端应在 3 秒内完成重连，且数据丢失率不超过 0.1%。

2.2 用户特征与角色

系统设计了三种核心角色，各角色拥有不同的权限边界与功能视图。

协作者（Collaborator） 具备发起绘画操作（画笔、橡皮擦、形状工具）、实时接收并渲染他人笔触、同步光标位置以及查看操作历史记录等核心能力。协作者无法清空他人绘制的笔触或修改房间基本属性；在观战模式下绘画权限可被管理员临时禁用。典型使用场景包括设计师团队实时共创、分组讨论中的白板标注等。

观战者 (Observer) 可以实时接收并渲染画布画面，以及查看操作历史记录。观战者无法发起任何绘画操作，且自身角色变更需经房间管理员审批。典型场景为远程会议旁听、设计评审观众席以及课堂教学观摩。

房间管理员 (Room Admin) 拥有协作者的全部能力，同时可执行画布清空操作（生成 Clear_Canvas 指令）、导出操作日志为 SVG/PNG、踢出违规用户以及设置房间属性（最大人数限制、只读模式开关）。房间管理员无法跨房间操作，也无法访问其他房间的管理员面板。典型场景为课堂教师掌控白板秩序、项目经理主持头脑风暴评审。

2.3 运行环境与技术栈约束

系统的部署与运行受到以下技术栈约束：

- **前端运行环境：**主流桌面浏览器（Chrome ≥ 90 、Firefox ≥ 88 、Edge ≥ 90 ）及移动端 Safari/Chrome。画布渲染层优先使用 HTML5 Canvas 2D API；对于复杂矢量图形，可引入 WebGL 加速渲染管线。
- **通信协议：**WebSocket API(RFC 6455)作为实时双向通信的首选协议；HTTP/1.1 Long Polling 作为降级备选。
- **后端运行时：**Go (Go ≥ 1.21) 或 Java (Spring Boot ≥ 3.0) 实现高性能 WebSocket 网关服务。
- **负载均衡层：**Nginx (≥ 1.20) 提供 TCP 层负载均衡，支持 WebSocket 协议的 Upgrade 与 Ping/Pong 机制。
- **缓存与消息总线：**Redis (≥ 6.2)，利用其 Pub/Sub 功能实现跨节点房间广播，List/ZSet 结构缓存近期操作流。
- **消息队列与持久化：**Apache Kafka (≥ 3.0)，用于操作日志的异步持久化与历史回放。
- **存储层：**MongoDB (≥ 5.0)，以文档结构存储矢量操作流，支持高效的范围查询与时间序列检索。

第三章 功能性需求

3.1 房间与会话管理域

3.1.1 房间创建与配置

- **F-RM-001: 房间创建。**认证用户通过 HTTP POST 接口 `/api/v1/rooms` 创建新房间，系统分配全局唯一房间 ID (Room_ID, UUID v4 格式)，并返回房间访问令牌 (Access Token)。
- **F-RM-002: 房间属性配置。**房间创建时可配置：房间名称（最大 64 字符）、最大协作者人数（默认 50，上限 200）、是否允许观战者进入、画布尺寸（默认 1920×1080，支持自定义分辨率）。
- **F-RM-003: 邀请码机制。**每个房间生成 6 位字母数字邀请码，用户可通过邀请码快捷加入房间，无需传递 Room_ID。

3.1.2 用户加入与路由分配

- **F-RM-004: 用户加入房间。**用户通过 WebSocket 连接 `/ws/room/{room_id}` 并携带身份令牌 (JWT) 加入房间。网关验证令牌后，将用户路由至对应房间的 Channel。
- **F-RM-005: 分布式路由分配。**Nginx 基于 IP_Hash 或 Cookie 粘性策略将用户分配至固定的 WebSocket 网关节点；Redis 中维护 Room_ID → Node_List 的映射关系，支持跨节点广播。
- **F-RM-006: 房间人数实时查询。**提供 HTTP GET 接口 `/api/v1/rooms/{room_id}/members`，返回当前在线用户数与用户基本信息列表。

3.2 全局画作同步域 (Snapshot & Replay) ——核心功能

3.2.1 新用户入房状态同步

- **F-GS-001: 历史操作流拉取。**新用户加入房间后, 前端首先调用 HTTP GET `/api/v1/rooms/{room_id}/operations` 接口, 传入客户端本地已知的最新操作序列号 `since_sequence_id` (首次加入时为 0)。后端返回该序列号之后的所有增量操作指令集 (Operation Log Segment)。
- **F-GS-002: 客户端本地回放引擎。**前端接收到操作指令集后, 在本地 Canvas 上按 `Sequence_ID` 升序逐条重放 (Replay) 指令, 还原当前画布状态。回放过程应对用户不可见 (后台静默执行)。
- **F-GS-003: 快照加速机制。**后端定期 (每 5 分钟或每新增 10,000 条操作) 对当前画布状态生成快照 (Snapshot), 存储于 MongoDB 的 `snapshots` 集合。新用户入房时, 若 `since_sequence_id` 早于最近快照时间戳, 后端优先加载快照, 再仅回放快照之后的增量操作, 显著缩短入房等待时间。

3.2.2 操作日志持久化

- **F-GS-004: 操作日志写入。**每条绘画操作在写入 Redis 缓存的同时, 异步发送至 Kafka 进行持久化。Kafka 消费者组将操作日志批量写入 MongoDB 的 `operations` 集合。
- **F-GS-005: 历史回放与导出。**提供 HTTP GET `/api/v1/rooms/{room_id}/export` 接口, 支持按时间范围导出操作日志为 SVG 或 PNG 格式。

3.3 实时协同绘画域 (Real-time Drawing) ——核心功能

3.3.1 笔触渲染

- **F-RD-001: 画笔工具捕获。**用户选择画笔工具后,前端监听 Canvas 的 mousedown、mousemove、mouseup (及对应的 touch 事件) 事件,捕获一系列贝塞尔曲线坐标点序列,封装为 Stroke 数据结构。
- **F-RD-002: 橡皮擦工具。**橡皮擦本质上是将背景色 (或透明色) 作为笔触颜色的特殊 Stroke,支持可配置的擦除半径。橡皮擦操作同样作为标准 Stroke 记录,不单独区分。
- **F-RD-003: 矢量路径压缩。**前端对捕获的坐标点进行 Douglas-Peucker 或 Ramer-Douglas-Peucker 路径简化算法处理,减少冗余中间点,降低网络传输量。

3.3.2 光标同步 (User Presence)

- **F-RD-004: 光标位置广播。**用户的鼠标/画笔悬停位置以独立的轻量级消息 (msg_type = cursor_sync) 实时广播至同房间其他用户,前端在对应坐标渲染用户光标图标与昵称标签。
- **F-RD-005: 光标节流。**客户端将光标位置以固定频率 (每 50ms 最多一次) 发送,避免高频 mousemove 事件 (每 16ms 触发一次) 造成的网络风暴。

3.3.3 操作广播

- **F-RD-006: 增量坐标包广播。**包含增量坐标点的 Stroke 数据包通过 WebSocket 实时广播给同房间的其他用户。广播采用服务器统一分配 Sequence_ID 的方式保证全局有序。
- **F-RD-007: 服务器端操作序列号分配。**每条到达服务器的绘画操作被赋予一个单调递增的 64 位无符号整数 Sequence_ID (由 Redis INCR 命令原子生成),确保所有客户端接收到的操作顺序完全一致。

3.3.4 功能需求点追踪

实时协同绘画域的功能需求以编号 F-RD-001 至 F-RD-010 逐条定义，具体内容如下：

F-RD-001（画笔工具捕获，P0 核心）：前端监听 Canvas 的鼠标和触摸事件序列，捕获贝塞尔曲线坐标点，封装为包含坐标数组、颜色和线宽的 Stroke 数据结构。F-RD-002（橡皮擦工具，P0 核心）：橡皮擦以背景色或透明色作为笔触颜色，支持可配置的擦除半径，与标准笔触同等记录。F-RD-003（矢量路径压缩，P1 重要）：前端对坐标点应用 Douglas-Peucker 或 Ramer-Douglas-Peucker 算法处理，减少冗余中间点以降低网络传输量。

F-RD-004（光标位置广播，P1 重要）：用户鼠标或画笔位置以独立轻量消息（`msg_type = cursor_sync`）实时广播至同房间其他用户，前端在对应坐标渲染光标图标与昵称标签。F-RD-005（光标节流，P1 重要）：客户端将光标位置以固定频率（每 50ms 最多一次）发送，将高频 `mousemove` 事件（每 16ms 触发一次）降至每秒不超过 20 条。

F-RD-006（增量坐标包广播，P0 核心）：Stroke 数据包通过 WebSocket 实时广播给同房间其他用户。F-RD-007（服务器端序列号分配，P0 核心）：每条到达服务器的绘画操作被赋予单调递增的 64 位无符号整数 `Sequence_ID`（由 Redis INCR 原子生成），确保所有客户端接收到的操作顺序完全一致。F-RD-008（笔触渲染，P0 核心）：客户端按 `Sequence_ID` 升序在 Canvas 上绘制矢量路径。F-RD-009（离线重连追平，P0 核心）：重连后客户端携带 `last_operation_id` 向服务器请求增量操作指令集，按序回放后恢复最新画布状态。F-RD-010（回放引擎，P1 重要）：支持历史操作日志流的任意倍速回放，精确重建任意历史时刻画布状态。

第四章 非功能性需求

4.1 性能与吞吐量需求 (Performance)

本节以可量化的性能指标 (KPI) 形式定义系统在性能维度上的约束条件。所有指标均在以下基准测试环境中测定：客户端位于同一地域数据中心，网络往返延迟 (RTT) 不超过 5ms；服务器配置为 16 核 CPU、32GB 内存、万兆网络。

并发接入能力。单房间最高支持不少于 200 名协作者同时在线（含绘画者与观战者），全局系统支持 10,000+ 并发 WebSocket 连接。单个 WebSocket 网关节点在 CPU 利用率不超过 70% 的条件下，应能稳定维持不少于 5,000 条并发连接。HTTP REST 房间管理接口的吞吐量应不低于 1,000 QPS。

实时性指标。笔触广播端到端延迟 P99 要求不超过 50ms, P50 不超过 20ms。端到端延迟定义为从发送端（用户 A）发出 stroke_move 消息，到接收端（用户 B）Canvas 完成渲染的时间。全局所有房间每秒处理的绘画操作消息总数应不低于 50,000 条。光标同步延迟 P99 不超过 100ms。

入房加载时间。新用户从 WebSocket 连接建立到 Canvas 完全渲染当前画布状态的耗时应不超过 2,000ms，包含快照加载与增量回放时间。

消息队列延迟。Kafka 从操作进入 Topic 到完全持久化至 MongoDB 的延迟不超过 200ms；同机房 Redis Cluster 内跨节点 Pub/Sub 消息传递的 P99 延迟不超过 5ms。

4.2 可用性与容错需求 (Availability & Fault Tolerance)

4.2.1 断线重连机制

- **NF-AFT-001: 自动重连策略。**客户端检测到 WebSocket 连接断开后, 应立即 (0 秒延迟) 尝试第一次重连; 若失败, 则以指数退避策略重试: 1s、2s、4s、8s、16s (最大 30s), 最多重试 10 次或重试总时长超过 5 分钟则放弃。
- **NF-AFT-002: Last_Operation_ID 追平机制。**重连成功后, 客户端向服务器发送 `sync_request` 消息, 携带本地已缓存的最新 `Sequence_ID`(`Last_Operation_ID`)。服务器从该 ID 之后的所有增量操作推送给客户端, 客户端按序回放后恢复至最新画布状态。理想情况下, 断线 5 秒内的数据丢失率不超过 0.1%。
- **NF-AFT-003: 乐观渲染与服务器校正。**客户端在断线期间应继续允许用户在本地画布上绘画 (乐观模式, Optimistic Update); 重连后, 服务器推送的确认序列将覆盖本地乐观状态。

4.2.2 节点故障隔离

- **NF-AFT-004: WebSocket 网关故障隔离。**当健康检查发现某个 WebSocket 网关节点不可用时, Nginx 自动将其从负载均衡池中移除。连接至该节点的用户将在下一次心跳超时 (默认 30s) 后触发重连流程, 被 Nginx 路由至健康的节点。
- **NF-AFT-005: Redis 节点故障。**Redis Cluster 中任意节点宕机时, 系统降级为读写分离模式: 写入操作路由至主节点, 房间广播降级为单节点内广播 (跨节点 Pub/Sub 中断), 直至 Sentinel 或 Cluster 自动故障转移完成。
- **NF-AFT-006: MongoDB 写入失败容错。**Kafka 的持久化消费链路发生故障时, 操作日志在 Redis 中保留 30 分钟 (TTL), 期间新连接仍可通过 Redis 缓存恢复画布状态。Kafka 恢复后, 积压消息 (Backlog) 由消费者组顺序消费补写。

第五章 接口设计规范

5.1 HTTP RESTful 接口

本节定义系统对外暴露的 HTTP REST 接口，采用 JSON 格式进行请求与响应。

房间管理接口。创建房间通过 POST `/api/v1/rooms` 实现，请求体包含 name (房间名称，最多 64 字符)、max_members (最大人数，默认 50，上限 200) 等字段，响应返回 room_id (UUID v4)、invite_code (6 位邀请码) 和 access_token (JWT 令牌)。获取房间详情通过 GET `/api/v1/rooms/{room_id}` 实现，返回房间元数据、当前在线人数及房间状态。查询房间成员列表通过 GET `/api/v1/rooms/{room_id}/members` 实现，返回所有在线成员的 user_id、role、光标位置及最后活跃时间。

操作同步接口。获取历史操作指令集通过 GET `/api/v1/rooms/{room_id}/operations` 实现，查询参数 since_sequence_id 指定起始序列号 (首次加入时为 0)，limit 指定最大返回条数 (默认 1000，上限 5000)，响应返回增量操作集与分页游标。获取最近快照通过 GET `/api/v1/rooms/{room_id}/snapshot` 实现，返回最近一次画布快照 (SVG 格式) 和对应的 sequence_id，用于新用户快速入房。

导出与管理接口。导出画作通过 GET `/api/v1/rooms/{room_id}/export` 实现，查询参数指定 format (svg/png)、start_seq、end_seq 和 dpi (仅 PNG)，返回下载链接或异步任务 ID。房间管理员可通过 POST `/api/v1/rooms/{room_id}/clear` 清空画布，该操作生成一条 operation_type=clear_canvas 的系统操作并广播至所有在线用户。

认证接口。用户登录通过 POST `/api/v1/auth/login` 实现，请求体包含 username 和 password，响应返回 JWT 令牌 (有效期 2 小时) 及刷新令牌 (有效期 7 天)。

5.2 WebSocket 事件交互协议

WebSocket 连接建立后, 客户端与服务器通过双向 JSON 消息进行实时交互。每条消息均遵循以下统一信封结构:

```
{  
  "msg_id": "uuid-v4",  
  "msg_type": "...",  
  "timestamp": 1700000000000,  
  "room_id": "...",  
  "user_id": "...",  
  "payload": { ... }  
}
```

5.2.1 上行事件 (Client → Server)

客户端通过 WebSocket 连接主动向服务器发送以下类型的事件:

笔触操作事件。 stroke_start 表示用户按下鼠标或触摸开始作画, payload 包含 color (十六进制颜色代码)、width (线宽像素) 和 tool (工具类型 brush/pencil/eraser)。stroke_move 表示笔触轨迹中的一系列坐标点, payload 包含 points 数组 (每个点包含 x、y 和 pressure 字段) 以及 stroke_id (用于关联同一笔触的三条记录)。stroke_end 表示用户释放鼠标或触摸, payload 包含 stroke_id 和 final_point。

状态同步事件。 cursor_sync 用于光标位置更新, payload 包含 x、y 坐标和当前工具类型, 客户端以固定频率 (每 50ms 最多一次) 发送以避免网络风暴。sync_request 用于重连后追平操作, payload 包含 last_operation_id (客户端本地已缓存的最新序列号)。

辅助事件。 ping 为心跳维持消息, 客户端每 25 秒无任何业务消息时主动发送, payload 包含时间戳。room_message 为文本聊天消息, payload 包含 text 字段。

5.2.2 下行事件 (Server → Client)

服务器主动向客户端推送以下类型的事件:

笔触广播事件。 stroke_update 推送来自他人的笔触数据, payload 包含 se-

quence_id (服务器分配的全局序列号)、user_id、stroke_id、type (move/end)、points 数组、color 和 width。canvas_cleared 由房间管理员触发, 通知所有用户清空画布, payload 包含 sequence_id 和 operator_id。

用户状态事件。 user_joined 通知新用户进入房间, payload 包含 user_id、nickname 和 role。user_left 通知用户退出房间, payload 仅包含 user_id。cursor_update 推送他人光标位置, payload 包含 user_id、x 和 y 坐标。

同步与辅助事件。 sync_response 在 sync_request 后推送追平数据, payload 包含 operations 数组和 latest_seq。pong 为服务器确认心跳, error 推送操作错误或权限异常信息 (包含 code 和 message 字段)。

5.2.3 心跳包 (Ping/Pong) 维持长连接机制

为防止中间设备 (如企业防火墙、Nginx) 对长时间空闲的 TCP 连接进行会话超时清理, 客户端需在无任何业务消息发送的情况下, **每 25 秒主动向服务器发送一条 ping 消息**。服务器收到后应立即回复 pong 消息。若服务器在 5 秒内未收到任何来自客户端的消息 (包括 ping), 服务器可主动关闭该连接, 以释放资源。

第六章 数据流与质量保障

6.1 数据完整性保障

- **序号连续性校验。**客户端在接收 `stroke_update` 消息时，应对 `Sequence_ID` 的连续性进行校验。若检测到跳号（如期望 1000，收到 1002），应立即向服务器发送 `sync_request` 追平遗漏操作，不得自行跳过或忽略。
- **幂等性设计。**客户端在乐观渲染时应为每条操作生成本地唯一操作 ID（Local Operation ID）。服务器推送的确认消息中若包含相同的 Local Operation ID，客户端应忽略重复渲染。
- **数据校验和。**对于超过一定长度（ $\geq 1,000$ 个坐标点）的 Stroke，客户端在发送前计算 CRC32 校验和并附加于 payload 中；服务器接收后校验，发现错误时丢弃并通知客户端重传。

6.2 安全性需求

- **NF-SEC-001：身份认证。**所有 WebSocket 连接必须携带有效的 JWT 令牌；JWT 中包含 `user_id`、`role` 与房间权限信息。
- **NF-SEC-002：房间访问控制。**用户只能加入持有邀请码或持有有效 Access Token 的房间。
- **NF-SEC-003：操作速率限制。**单个用户在任意 1 秒窗口内发送的绘画消息不得超过 60 条，超出部分服务器返回 `RATE_LIMIT_EXCEEDED` 错误。
- **NF-SEC-004：输入校验。**所有坐标点必须落在画布分辨率范围内（ $[0, width] \times [0, height]$ ），超范围数据将被服务器截断至边界值。